Universidad Nacional

Sede Regional Brunca – Campus Coto

BlaxtorGym

Plan de desarrollo modular actualizado

Fases, dependencias, archivos, estado actual y estructura completa

Integrantes: Justin Rojas Jarquin
Cristopher Ureña Valverde

Tecnologías: C#, WPF, .NET 10, SQLite, Entity Framework Core

Estrategia actual: primero funcionalidad, persistencia, reglas de negocio e integración. El diseño visual definitivo se realiza cuando los módulos principales estén funcionando.

Documento de seguimiento técnico del proyecto

Índice

1. Cómo utilizar este documento **2**

2. Punto actual del desarrollo **2**

3. Mapa general de fases **2**

4. Especificación detallada por fase **3**
4.1. Fase 1 – Configuración inicial del proyecto . . . 3
4.2. Fase 2 – Persistencia: SQLite + Entity Framework Core . . . 3
4.3. Fase 3 – Autenticación / Login . . . 4
4.4. Fase 4 – Tipos de cliente . . . 5
4.5. Fase 5 – Clientes . . . 5
4.6. Fase 6 – Tipos de membresía + Tarifas . . . 6
4.7. Fase 7 – Mediciones físicas . . . 6
4.8. Fase 8 – Membresías . . . 7
4.9. Fase 9 – Pagos . . . 7
4.10. Fase 10 – Notificaciones de vencimiento . . . 7
4.11. Fase 11 – Backups . . . 8
4.12. Fase 12 – Reportes / PDF . . . 8
4.13. Fase 13 – Bitácora de cambios . . . 9
4.14. Fase 14 – Integración general . . . 9
4.15. Fase 15 – Diseño visual definitivo . . . 9
4.16. Fase 16 – Pruebas y optimización final . . . 10

5. Ruta crítica y trabajo en paralelo **10**

6. Principios técnicos obligatorios **10**

7. Definition of Done general **11**

8. Regla para archivos nuevos **11**

Cómo utilizar este documento

Este documento actualiza el plan original y refleja el avance real del proyecto. Mantiene la convención de una sola aplicación WPF con capas lógicas dentro del mismo proyecto. Se conserva el nombre físico `BlaxtorGym.Infraestructure` para evitar cambios de namespace durante el desarrollo.

Convención de estado
✓ = implementado y probado. ▶ = en desarrollo o preparado para comenzar. □ = pendiente.

Los archivos nuevos se agregan únicamente cuando tienen una responsabilidad real. Si durante fases futuras aparece una necesidad concreta, el plan puede incorporar nuevos archivos sin forzar abstracciones innecesarias.

Punto actual del desarrollo

El proyecto ya completó la configuración inicial, la persistencia con SQLite/EF Core, la autenticación y el módulo de tipos de cliente. El flujo funcional actual es:

```
WPF ->Login ->Services ->Repositories ->EF Core ->SQLite ->MainWindow
                                    ->TipoClienteView
```

Se añadió infraestructura reutilizable que no estaba en la primera versión del plan: seeding, comandos MVVM y un sistema centralizado de notificaciones. El diseño visual final continúa pospuesto deliberadamente.

Mapa general de fases

Estado	Responsable	Fase / módulo	Dependencias
✓	Justin	1. Configuración inicial	Ninguna
✓	Justin	2. SQLite + EF Core	Fase 1
✓	Justin	3. Autenticación / Login	Fase 2
✓	Justin	4. Tipos de cliente	Fase 2
▶	Cristopher	5. Clientes	Fases 2 y 4
▶	Justin	6. Tipos de membresía + Tarifas	Fases 2 y 4
□	Cristopher	7. Mediciones físicas	Fase 5
□	Justin	8. Membresías	Fases 5 y 6
□	Justin	9. Pagos	Fase 8
□	Cristopher	10. Notificaciones de vencimiento	Fases 5 y 8
□	Cristopher	11. Backups	Fase 2 estable
□	Cristopher	12. Reportes / PDF	Fases 5, 7, 8 y 9
□	Ambos	13. Bitácora de cambios	Módulos principales
□	Ambos	14. Integración general	Fases 1–13
□	Ambos	15. Diseño visual definitivo	Fase 14
□	Ambos	16. Pruebas y optimización final	Fases 1–15

Especificación detallada por fase

Fase 1 – Configuración inicial del proyecto

Estado: ✓ Completada **Responsable:** Justin
Dependencias: Ninguna

Objetivo. Dejar una única aplicación WPF compilable, con paquetes, carpetas lógicas y convenciones listas para desarrollar.

Archivos y estructura principales

- ✓ `BlaxtorGym.csproj`
- ✓ `App.xaml`
- ✓ `App.xaml.cs`
- ✓ `MainWindow.xaml`
- ✓ `MainWindow.xaml.cs`
- ✓ Carpetas lógicas `Application`, `Domain`, `Infraestructure` y `WPF`

Resultado. La solución compila en limpio y permite avanzar módulo por módulo sin dividir el proyecto en múltiples ensamblados.

Fase 2 – Persistencia: SQLite + Entity Framework Core

Estado: ✓ Completada **Responsable:** Justin
Dependencias: Fase 1

Objetivo. Construir el modelo persistente completo, aplicar migraciones y asegurar la creación automática de `blaxtorgym.db`.

Dominio implementado

- ✓ `src/BlaxtorGym.Domain/Common/BaseEntity.cs`
- ✓ `src/BlaxtorGym.Domain/Enums/RolUsuario.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Usuario.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/TipoCliente.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Cliente.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/TipoMembresia.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Precio.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Membresia.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Pago.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/MedicionFisica.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/Notificacion.cs`
- ✓ `src/BlaxtorGym.Domain/Entities/BitacoraCambio.cs`

Persistencia implementada

- ✓ `src/BlaxtorGym.Infraestructure/Data/AppDbContext.cs`
- ✓ `src/BlaxtorGym.Infraestructure/Data/DatabaseInitializer.cs`
- ✓ `src/BlaxtorGym.Infraestructure/Data/DatabaseSeeder.cs` *(nuevo)*
- ✓ `src/BlaxtorGym.Infraestructure/Configuration/DatabaseSettings.cs`
- ✓ `Migrations/InitialCreate.cs` y snapshot de EF Core

Comprobaciones completadas

- ✓ Tablas, relaciones, restricciones e índices creados correctamente.
- ✓ Migraciones aplicadas con `MigrateAsync()`.
- ✓ Seeding del administrador inicial con contraseña BCrypt.
- ✓ Base SQLite recreable desde cero.
- ✓ Integridad de claves foráneas verificada.

Fase 3 – Autenticación / Login

Estado: ✓ Completada funcionalmente **Responsable:** Justin
Dependencias: Fase 2

Objetivo. Implementar HU-01 con BCrypt, usuario activo/inactivo, intentos fallidos, bloqueo temporal y navegación posterior al acceso correcto.

Archivos implementados

- ✓ `src/BlaxtorGym.Domain/Interfaces/IUsuarioRepository.cs`
- ✓ `src/BlaxtorGym.Infraestructure/Repositories/UsuarioRepository.cs`
- ✓ `src/BlaxtorGym.Application/DTOs/LoginRequestDto.cs`
- ✓ `src/BlaxtorGym.Application/DTOs/ResultadoLogin.cs` *(nuevo)*
- ✓ `src/BlaxtorGym.Application/Services/AutenticacionService.cs`
- ✓ `src/BlaxtorGym.WPF/Commands/RelayCommand.cs` *(nuevo, reutilizable)*
- ✓ `src/BlaxtorGym.WPF/ViewModels/LoginViewModel.cs`
- ✓ `src/BlaxtorGym.WPF/Views/LoginWindow.xaml`
- ✓ `src/BlaxtorGym.WPF/Views/LoginWindow.xaml.cs`
- ✓ `src/BlaxtorGym.WPF/Notifications/TipoNotificacion.cs` *(nuevo)*
- ✓ `src/BlaxtorGym.WPF/Notifications/INotificationService.cs` *(nuevo)*
- ✓ `src/BlaxtorGym.WPF/Notifications/NotificationServiceBase.cs` *(nuevo)*
- ✓ `src/BlaxtorGym.WPF/Notifications/MessageBoxNotificationService.cs` *(nuevo)*
- ✓ `App.xaml.cs` y conexión temporal con `MainWindow`

Escenarios comprobados

- ✓ Login exitoso.
- ✓ Usuario inexistente.
- ✓ Contraseña incorrecta.
- ✓ Usuario inactivo.

- ✓ Bloqueo temporal tras tres intentos fallidos.
- ✓ Navegación a `MainWindow` tras autenticación correcta.
- ✓ Notificaciones centralizadas, reemplazables posteriormente por Snackbar/Toast.

Fase 4 – Tipos de cliente

Estado: ✓ Completada funcionalmente **Responsable:** Justin
Dependencias: Fase 2

Objetivo. Administrar el catálogo de tipos de cliente y dejarlo disponible para Clientes y Tarifas.

Archivos implementados

- ✓ `src/BlaxtorGym.Domain/Interfaces/ITipoClienteRepository.cs`
- ✓ `src/BlaxtorGym.Infraestructure/Repositories/TipoClienteRepository.cs`
- ✓ `src/BlaxtorGym.Application/Services/TipoClienteService.cs`
- ✓ `src/BlaxtorGym.WPF/ViewModels/TipoClienteViewModel.cs`
- ✓ `src/BlaxtorGym.WPF/Views/TipoClienteView.xaml`
- ✓ `src/BlaxtorGym.WPF/Views/TipoClienteView.xaml.cs`
- ✓ Integración temporal en `MainWindow.xaml` y `MainWindow.xaml.cs`

Comportamientos comprobados

- ✓ Listar activos e inactivos.
- ✓ Crear y modificar tipos de cliente.
- ✓ Evitar nombres duplicados.
- ✓ Activar y desactivar sin borrar historial.
- ✓ Consultas de lectura con `AsNoTracking()` cuando corresponde.
- ✓ Notificaciones reutilizables para éxito, advertencia y error.

Fase 5 – Clientes

Estado: ▶ Preparada para Cristopher **Responsable:** Cristopher
Dependencias: Fases 2 y 4

Objetivo. Implementar registro, consulta, modificación y desactivación de clientes.

Archivos a crear

- ☐ `src/BlaxtorGym.Domain/Interfaces/IClienteRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/ClienteRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/ClienteService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ClienteViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ClienteView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/ClienteView.xaml.cs`

Nota. No se fuerza un DTO adicional en esta fase. Si durante la implementación aparece una necesidad real de separar entrada/salida, se añadirá en ese momento.

Tareas

- ☐ Validar cédula, nombre, fecha de nacimiento y datos de contacto.
- ☐ Asociar correctamente un `TipoCliente`.
- ☐ Evitar cédulas duplicadas.
- ☐ Modificar sin perder relaciones históricas.
- ☐ Desactivar clientes en vez de eliminarlos.
- ☐ Preparar filtros/paginación para crecimiento futuro.

Fase 6 – Tipos de membresía + Tarifas

Estado: ▶ Puede iniciar en paralelo **Responsable:** Justin
Dependencias: Fases 2 y 4

Objetivo. Gestionar la duración de membresías y los precios por combinación de tipo de cliente y tipo de membresía.

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/ITipoMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Domain/Interfaces/IPrecioRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/TipoMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/PrecioRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/TipoMembresiaService.cs`
- ☐ `src/BlaxtorGym.Application/Services/TarifaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/TarifasViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/TarifasView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/TarifasView.xaml.cs`

Tareas

- ☐ Administrar semanal, quincenal y mensual con duración en días.
- ☐ Administrar precios por combinación TipoCliente + TipoMembresia.
- ☐ Evitar combinaciones duplicadas.
- ☐ Mantener vigencia e historial de tarifas.
- ☐ Consultar eficientemente la tarifa aplicable.

Fase 7 – Mediciones físicas

Estado: ☐ Pendiente **Responsable:** Cristopher
Dependencias: Fase 5

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IMedicionFisicaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/MedicionFisicaRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/MedicionFisicaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/MedicionesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/MedicionesView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/MedicionesView.xaml.cs`

Fase 8 – Membresías

Estado: ☐ Pendiente **Responsable:** Justin
Dependencias: Fases 5 y 6

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/MembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/MembresiaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/MembresiasViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/MembresiasView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/MembresiasView.xaml.cs`

Objetivo. Crear y renovar membresías asociando cliente, tipo de membresía y tarifa aplicada, conservando historial.

Fase 9 – Pagos

Estado: ☐ Pendiente **Responsable:** Justin
Dependencias: Fase 8

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IPagoRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/PagoRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/PagoService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/PagosViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/PagosView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/PagosView.xaml.cs`

Fase 10 – Notificaciones de vencimiento

Estado: ☐ Pendiente **Responsable:** Cristopher
Dependencias: Fases 5 y 8

Nota. Esta fase corresponde a notificaciones de negocio por vencimiento. Es distinta de la infraestructura visual `WPF/Notifications` ya creada para mensajes de la interfaz.

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/INotificacionRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/NotificacionRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/NotificacionService.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Services/CorreoService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/NotificacionesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/NotificacionesView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/NotificacionesView.xaml.cs`

Fase 11 – Backups

Estado: ☐ Pendiente **Responsable:** Cristopher
Dependencias: Fase 2 estable

Objetivo. Respaldar y restaurar SQLite de forma controlada, sin congelar WPF y validando el archivo antes de restaurar.

Archivos previstos

- ☐ `src/BlaxtorGym.Application/Services/BackupService.cs` *(si se mantiene la abstracción a nivel Application)*
- ☐ `src/BlaxtorGym.Infraestructure/Services/BackupService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ConfiguracionViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ConfiguracionView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/ConfiguracionView.xaml.cs`

Fase 12 – Reportes / PDF

Estado: ☐ Pendiente **Responsable:** Cristopher
Dependencias: Fases 5, 7, 8 y 9

Archivos previstos

- ☐ `src/BlaxtorGym.Application/DTOs/Reporte*.cs`
- ☐ `src/BlaxtorGym.Application/Services/ReporteService.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Services/PdfService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ReportesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ReportesView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/ReportesView.xaml.cs`

Fase 13 – Bitácora de cambios

Estado: ☐ Pendiente **Responsable:** Ambos
Dependencias: Módulos principales funcionales

Archivos previstos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IBitacoraRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/BitacoraRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/BitacoraService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/BitacoraViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/BitacoraView.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/BitacoraView.xaml.cs`

Fase 14 – Integración general

Estado: ☐ Pendiente **Responsable:** Ambos
Dependencias: Fases 1–13

Objetivo. Reemplazar conexiones temporales por navegación centralizada, consolidar dependencias y asegurar un flujo completo de extremo a extremo.

Archivos previstos o a revisar

- ☐ `App.xaml`
- ☐ `App.xaml.cs`
- ☐ `MainWindow.xaml`
- ☐ `MainWindow.xaml.cs`
- ☐ `src/BlaxtorGym.WPF/NavigationService.cs` *(crear solo si la navegación central realmente lo requiere)*
- ☐ ViewModels y Services que requieran ajustes de integración.

Fase 15 – Diseño visual definitivo

Estado: ☐ Pendiente **Responsable:** Ambos
Dependencias: Fase 14

Objetivo. Aplicar identidad visual y UX sin modificar reglas de negocio.

Elementos previstos

- ☐ Menú lateral en `MainWindow`.
- ☐ Contenedor central que cambia entre `UserControl` por módulo.
- ☐ Transiciones suaves (fade/slide) y estados de selección.
- ☐ Estilos centralizados en `src/BlaxtorGym.WPF/Resources/*.xaml`.
- ☐ Sustituir `MessageBoxNotificationService` por notificaciones visuales modernas si conviene.

- ☐ Iconos, tipografía, estados de carga, vacío y error.
- ☐ Virtualización de listas grandes.

Fase 16 – Pruebas y optimización final

Estado: ☐ Pendiente **Responsable:** Ambos
Dependencias: Todas las fases anteriores

Objetivo. Validar requisitos, regresiones, rendimiento y estabilidad con crecimiento de datos.

- ☐ Ejecutar casos funcionales por módulo.
- ☐ Probar entradas inválidas y escenarios de excepción.
- ☐ Medir consultas y tiempos con volúmenes crecientes.
- ☐ Revisar índices, `AsNoTracking()`, paginación y N+1.
- ☐ Confirmar que el I/O no congele WPF.
- ☐ Optimizar únicamente cuellos de botella medidos.

Ruta crítica y trabajo en paralelo

Justin: Configuración → Persistencia → Login → Tipos de cliente → Tipos de membresía/Tarifas → Membresías → Pagos.
Cristopher: Clientes → Mediciones → Notificaciones → Backups → Reportes/PDF.

La Fase 6 puede avanzar en paralelo con Clientes. La Fase 8 sí debe esperar a que Clientes y Tarifas estén funcionales.

Principios técnicos obligatorios

Principio	Aplicación en BlaxtorGym
SOLID pragmático	Una clase debe tener responsabilidad clara. Interfaces solo cuando aporten desacoplamiento real.
MVVM	View presenta; ViewModel maneja estado/comandos; Service ejecuta reglas; Repository persiste.
EF Core / SQLite	Filtrar, ordenar y proyectar en SQLite; usar `AsNoTracking()` en lectura; evitar N+1; mantener índices útiles.
Async/Await	No usar `.Result` ni `.Wait()` en WPF. El I/O no debe congelar la interfaz.
Historial	No destruir información necesaria. Clientes se desactivan; tarifas, membresías y pagos conservan trazabilidad.
Notificaciones UI	ViewModels llaman a `INotificationService`; la implementación visual puede cambiar sin reescribir módulos.
Errores	No usar `catch` vacíos. Distinguir resultados de negocio de excepciones inesperadas.

Seguridad	Contraseñas con BCrypt; no almacenar texto plano. Considerar cifrado/backup seguro antes de entrega.
Diseño	Primero funcionalidad; estilos, efectos, iconos y navegación visual definitiva en Fase 15.

Definition of Done general

Antes de marcar una fase como completada:

- ☐ El código compila sin errores.
- ☐ La funcionalidad principal fue probada.
- ☐ Los datos se persisten correctamente cuando corresponde.
- ☐ Las entradas inválidas son rechazadas de forma controlada.
- ☐ Los errores inesperados no cierran la aplicación sin manejo.
- ☐ La interfaz no se bloquea durante operaciones costosas.
- ☐ Se mantiene la separación View – ViewModel – Service – Repository.
- ☐ No se rompieron módulos previamente funcionales.
- ☐ Se ejecutaron las pruebas principales del módulo.

Regla para archivos nuevos

El plan no pretende adivinar todos los archivos futuros. Si una fase requiere un archivo no previsto, se agregará únicamente cuando exista una necesidad concreta y se indicará su ruta, responsabilidad y motivo antes de crearlo. Esto evita tanto el sobre-diseño como el crecimiento desordenado del proyecto.